

ODOO DEVELOPER PROGRAM

Sequences, Data & Email Templates

Odoo 17 Development Tutorials

Ahmed Muhumed

July 26, 2026

OUTLINE

- 1 Introduction
- 2 Sequences
- 3 XML Data Importing
- 4 CSV Data Importing
- 5 Demo Data
- 6 Email Templates
- 7 Summary

Introduction

WHAT WILL WE LEARN?

- ▶ **Sequences** – automatic running numbers (Student/001, INV/2026/0001, ...)
- ▶ **XML data importing** – loading master/config data with modules
- ▶ **CSV data importing** – bulk tabular data via CSV files
- ▶ **Demo data** – sample records for training and testing databases
- ▶ **Email templates** – reusable QWeb mail content for notifications

WHY THESE TOPICS MATTER

- ▶ Real Odoo modules need default data, not only Python models.
- ▶ Sequences give professional, unique document numbers.
- ▶ XML/CSV data make installs repeatable.
- ▶ Demo data helps trainers and developers test quickly.
- ▶ Email templates standardize communication with users/customers.

Sequences

WHAT IS A SEQUENCE?

- ▶ A **sequence** generates the next number for a document or record.
- ▶ In Odoo, sequences are stored in `ir.sequence`.
- ▶ Examples:
 - Students: STU/0001
 - Invoices: INV/2026/00012
 - Tickets: TKT/78

Core Idea

Python asks the sequence for the next value, then stores it on the record.

DEFINING A SEQUENCE IN XML

```
1 <record id="seq_student_student" model="ir.sequence">
2   <field name="name">Student Sequence</field>
3   <field name="code">student.student</field>
4   <field name="prefix">STU/</field>
5   <field name="padding">4</field>
6   <field name="number_next">1</field>
7   <field name="number_increment">1</field>
8   <field name="company_id" eval="False"/>
9 </record>
```

- ▶ code is how Python finds the sequence.
- ▶ padding controls leading zeros (0001).

BREAKING DOWN SEQUENCE FIELDS

- ▶ **name**: label shown in Settings
- ▶ **code**: technical key used in `next_by_code()`
- ▶ **prefix / suffix**: text around the number
- ▶ **padding**: number width
- ▶ **number_next**: next integer to use
- ▶ **number_increment**: usually 1
- ▶ **company_id**: set for per-company sequences

USING A SEQUENCE IN PYTHON

Method Syntax

```
1 @api.model.create_multi
2 def create(self, vals_list):
3     for vals in vals_list:
4         if not vals.get('code'):
5             vals['code'] = self.env['ir.sequence'].next_by_code(
6                 'student.student'
7             ) or '/'
8     return super().create(vals_list)
```

- ▶ `next_by_code('student.student')` returns the next formatted value.
- ▶ If the sequence is missing, it returns `False`.

DATE-AWARE PREFIXES

```
1 <field name="prefix">STU/%(year)s/</field>  
2 <field name="padding">5</field>  
3 <field name="use_date_range">True</field>
```

► Common placeholders:

- %(year)s
- %(month)s
- %(day)s
- %(doy)s

► use_date_range helps restart numbering by period when configured.

PRACTICAL STUDENT EXAMPLE

```
1 code = fields.Char(string='Student Code', copy=False, readonly=True)
2
3 @api.model.create_multi
4 def create(self, vals_list):
5     seq = self.env['ir.sequence']
6     for vals in vals_list:
7         if vals.get('code'):
8             continue
9         vals['code'] = seq.next_by_code('student.student') or '/'
10    return super().create(vals_list)
```

- Keep generated codes readonly and copy=False.

SEQUENCE BEST PRACTICES

- ▶ One business document type → one sequence code.
- ▶ Never hard-code incremental numbers in Python.
- ▶ Decide early if the sequence is global or per company.
- ▶ Put sequence XML in `data/` and list it in the manifest.
- ▶ Protect important codes with uniqueness constraints when needed.

XML Data Importing

WHAT IS XML DATA IMPORTING?

- ▶ XML data files load records when a module is installed or updated.
- ▶ Perfect for:
 - configuration data
 - sequences
 - email templates
 - menus/actions/security
 - master data that must exist in every database

Manifest Entry

```
1 'data': [  
2     'data/ir-sequence-data.xml',  
3     'data/student-data.xml',  
4 ],
```

BASIC XML RECORD STRUCTURE

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <odoo>
3   <record id="student_demo_ahmed" model="student.student">
4     <field name="name">Ahmed Muhumed</field>
5     <field name="email">ahmed@example.com</field>
6     <field name="age">22</field>
7     <field name="gender">male</field>
8   </record>
9 </odoo>
```

- ▶ `id`: XML / external ID inside the module.
- ▶ `model`: target model technical name.

BREAKING DOWN IMPORTANT XML FEATURES

```
1 <!-- Many2one by XML ID -->
2 <field name="classroom_id" ref="classroom_grade_12"/>
3
4 <!-- Boolean / numbers with eval -->
5 <field name="active" eval="True"/>
6 <field name="age" eval="18"/>
7
8 <!-- Search existing record -->
9 <field name="country_id" search="['code', '=', 'SO']"/>
```

- ▶ **ref**: link to another XML ID.
- ▶ **eval**: Python expression for non-string values.
- ▶ **search**: find a record by domain.

NOUPDATE AND FORCECREATE

```
1 <odoo noupdate="1">
2   <record id="seq_student_student" model="ir.sequence">
3     <field name="name">Student Sequence</field>
4     <field name="code">student.student</field>
5     <field name="prefix">STU/</field>
6     <field name="padding">4</field>
7   </record>
8 </odoo>
```

- ▶ `noupdate="1"`: do not overwrite the record on later module updates.
- ▶ Useful for sequences and data users may customize.

FUNCTION CALLS AND DELETE IN XML

```
1 <function model="student.student" name="action_activate_all"/>  
2  
3 <delete model="student.student" search="[( 'name', '=', 'Temporary') ]"/>
```

- ▶ `<function>` can call a model method during loading.
- ▶ `<delete>` removes matching records.
- ▶ Use carefully; prefer normal records for most data.

XML DATA LOADING ORDER

```
1 'data': [  
2     'security/school_security.xml',  
3     'security/ir.model.access.csv',  
4     'data/ir_sequence_data.xml',  
5     'data/classroom_data.xml',  
6     'data/student_data.xml',  
7     'views/student_views.xml',  
8 ],
```

- ▶ Load dependencies first (security, parents, referenced records).
- ▶ A wrong order causes missing XML ID / access errors.

XML DATA BEST PRACTICES

- ▶ Use stable, descriptive XML IDs.
- ▶ Keep one concern per file when possible.
- ▶ Use `noupdate="1"` for user-customizable config.
- ▶ Prefer `ref` over hard-coded database IDs.
- ▶ Always list files in `__manifest__.py`.

CSV Data Importing

WHAT IS CSV DATA IMPORTING?

- ▶ CSV files can also load records during module install/update.
- ▶ Best for tabular, repetitive data such as:
 - access rights (`ir.model.access.csv`)
 - simple master lists
 - bulk demo rows

Manifest Entry

```
1 'data': [  
2     'security/ir.model.access.csv',  
3     'data/student_level_data.csv',  
4 ],
```

CSV FILE STRUCTURE

```
1 id , name , code , active  
2 student_level_beginner , Beginner , BEG , True  
3 student_level_intermediate , Intermediate , INT , True  
4 student_level_advanced , Advanced , ADV , True
```

- ▶ First row = field names.
- ▶ *id* column = XML / external ID (without module prefix in file).
- ▶ Other columns map to model fields.

HOW ODOO KNOWS THE MODEL

- ▶ For most CSV files, the model is deduced from the filename / import conventions used by the module loader.
- ▶ Classic and most common case:

```
1 security/ir.model.access.csv
```

- ▶ Access rights CSV targets `ir.model.access`.

ACCESS RIGHTS CSV EXAMPLE

```
1 id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
2 access_student_user,student.user,model_student_student,base.group_user,1,0,0,0
3 access_student_manager,student.manager,model_student_student,school.
   group_school_manager,1,1,1,1
```

- ▶ `model_id:id` and `group_id:id` use XML IDs.
- ▶ 1/0 mean True/False for permissions.

RELATIONAL VALUES IN CSV

```
1 id ,name, classroom_id/id ,gender  
2 student_ahmed ,Ahmed Muhumed, classroom_grade_12 ,male  
3 student_amina ,Amina Ali ,classroom_grade_12 ,female
```

- ▶ `classroom_id/id` writes a many2one through an XML ID.
- ▶ This is cleaner than numeric database IDs.

XML vs CSV

- ▶ Use **XML** when you need:
 - complex structures
 - noupdate, function, rich field options
 - menus/views/templates
- ▶ Use **CSV** when you need:
 - simple bulk rows
 - access rights tables
 - easy spreadsheet editing

CSV BEST PRACTICES

- ▶ Keep headers exact field names.
- ▶ Always provide an `id` column for stable external IDs.
- ▶ Prefer `field/id` for many2one relations.
- ▶ Validate CSV in a clean database install.
- ▶ Do not mix too much complex logic into CSV; switch to XML when needed.

Demo Data

WHAT IS DEMO DATA?

- ▶ **Demo data** is sample data loaded only when the database is created with demo enabled.
- ▶ Ideal for training, screenshots, and local testing.

Manifest Entry

```
1 'demo': [  
2   'demo/classroom_demo.xml',  
3   'demo/student_demo.xml',  
4 ] ,
```

- ▶ Demo files are **not** loaded in normal production installs without demo.

DEMO VS DATA

- ▶ data: required for the module to work (security, sequences, core config).
- ▶ demo: optional samples for learning/testing.

```
1 'data': [  
2     'security/ir.model.access.csv',  
3     'data/ir_sequence_data.xml',  
4 ],  
5 'demo': [  
6     'demo/student_demo.xml',  
7 ],
```

DEMO XML EXAMPLE

```
1 <odoo>
2   <record id="demo_classroom.a" model="school.classroom">
3     <field name="name">Grade 12-A</field>
4   </record>
5
6   <record id="demo_student.ahmed" model="student.student">
7     <field name="name">Ahmed Muhumed</field>
8     <field name="email">ahmed@example.com</field>
9     <field name="classroom_id" ref="demo_classroom.a"/>
10   </record>
11 </odoo>
```

DEMO CSV EXAMPLE

```
1 id ,name, age , gender , classroom_id/id  
2 demo_student_amina ,Amina Ali ,21 ,female ,demo_classroom_a  
3 demo_student_omar ,Omar Hassan ,19 ,male ,demo_classroom_a
```

- ▶ Demo can be XML, CSV, or both.
- ▶ Keep demo IDs prefixed with `demo_` for clarity.

WHEN TO USE DEMO DATA

- ▶ Training databases for students.
- ▶ Local developer testing without manual form filling.
- ▶ Showcasing an app with realistic sample records.
- ▶ UI/view testing with enough records for kanban/graph/pivot.

DEMO DATA BEST PRACTICES

- ▶ Never put required production config only in demo.
- ▶ Keep demo realistic but small and understandable.
- ▶ Use clear names and emails.
- ▶ Make demo depend on already loaded models/security.
- ▶ Recreate the DB with demo checked to verify loading.

Email Templates

WHAT IS AN EMAIL TEMPLATE?

- ▶ An **email template** is a reusable mail design stored in `mail.template`.
- ▶ It can render subject/body with QWeb and record values.
- ▶ Used for welcome mails, resets, invoices, reminders, and notifications.

Model

1 `mail.template`

DEFINING AN EMAIL TEMPLATE IN XML

```
1 <record id="mail_template_student_welcome" model="mail_template">
2   <field name="name">Student: Welcome Email</field>
3   <field name="model_id" ref="model_student_student"/>
4   <field name="subject">Welcome ${object.name}</field>
5   <field name="email_from">{{ (object.company_id.email or user.email) }}</field>
6   <field name="email_to">{{ object.email }}</field>
7   <field name="body_html" type="html">
8     <p>Hello <t t-out="object.name"/>,</p>
9     <p>Welcome to our school system.</p>
10  </field>
11 </record>
```

BREAKING DOWN TEMPLATE FIELDS

- ▶ **name**: template label
- ▶ **model_id**: model this template applies to
- ▶ **subject**: email subject (dynamic)
- ▶ **email_from** / **email_to**: addresses
- ▶ **body_html**: HTML body, often QWeb
- ▶ Optional: `report_template_ids`, attachments, language, etc.

SENDING A TEMPLATE FROM PYTHON

Method Syntax

```
1 def action_send_welcome_email(self):  
2     template = self.env.ref(  
3         'school_manager.mail_template_student_welcome'  
4     )  
5     for student in self:  
6         template.send_mail(student.id, force_send=True)  
7     return True
```

- ▶ `send_mail(res_id)` renders and sends for that record.
- ▶ `force_send=True` sends immediately instead of only queueing.

QWEB IN EMAIL BODIES

```
1 <field name="body_html" type="html">
2   <div>
3     <p>Dear <t t-out="object.name"/>,</p>
4     <p>Your code is
5       <strong><t t-out="object.code"/></strong>.
6     </p>
7     <p t-if="object.classroom_id">
8       Classroom:
9       <t t-out="object.classroom_id.name"/>
10    </p>
11  </div>
12 </field>
```

- ▶ `object` is the current record.
- ▶ Use `t-out` in modern Odoo QWeb.

OPENING COMPOSER WITH A TEMPLATE

```
1 def action_share_by_email(self):  
2     self.ensure_one()  
3     template = self.env.ref(  
4         'school_manager.mail_template_student_welcome'  
5     )  
6     return {  
7         'type': 'ir.actions.act_window',  
8         'name': 'Send Email',  
9         'res_model': 'mail.compose.message',  
10        'view_mode': 'form',  
11        'target': 'new',  
12        'context': {  
13            'default_model': 'student.student',  
14            'default_res_ids': self.ids,  
15            'default_template_id': template.id,  
16            'default_composition_mode': 'comment',  
17        },  
18    }
```

EMAIL TEMPLATE BEST PRACTICES

- ▶ Keep one template per clear business event.
- ▶ Use XML IDs and `env.ref()` instead of hard-coded IDs.
- ▶ Prefer QWeb dynamic fields over hardcoded names.
- ▶ Test rendering with real demo records.
- ▶ Put templates under `data/` and load them from the manifest.

Summary

- ▶ **Sequences** generate professional running numbers through `ir.sequence`.
- ▶ **XML data** loads structured module records and configuration.
- ▶ **CSV data** is ideal for bulk tabular rows and access rights.
- ▶ **Demo data** provides optional sample records for training/testing.
- ▶ **Email templates** standardize dynamic outgoing messages with QWeb.

PRACTICAL CHECKLIST

- ▶ Need STU/0001? → `create ir.sequence + next_by_code()`
- ▶ Need install-time master data? → **XML** in data
- ▶ Need bulk simple rows / ACL? → **CSV**
- ▶ Need sample training records? → **files** in demo
- ▶ Need welcome/reminder mail? → `mail.template + sendmail()`

SUGGESTED MODULE LAYOUT

```
1 school_manager/  
2   data/  
3     ir_sequence_data.xml  
4     mail_template_data.xml  
5   demo/  
6     student_demo.xml  
7   security/  
8     ir.model.access.csv  
9   models/  
10    student_student.py  
11    __manifest__.py
```